SENSEI ANALYZER SYSTEM

Real-Time Log Analysis with LLM-Powered Intent Detection

Field	Details
Project Name	Sensei Analyzer System
Document Type	Internal Technical Configuration & Deployment Guide
Version	1.0
Date	April 7, 2026
Status	Production Ready
Classification	Confidential – Internal Use Only
Repository Root	agentcore-typescript /
AWS Region	us-east-1 (N. Virginia)
AI Model	Claude Sonnet 4.6 via AWS Bedrock Inference Profile

1. Project Overview

Sensei Analyzer is a real-time log analysis system that uses LLM-powered intent detection to intelligently analyze agent execution traces, evaluator metrics, and X-Ray distributed trace data. Users ask natural language questions and the system automatically determines what data to fetch and how to present it — with no query syntax or schema knowledge required.

1.1 Core Technology Stack

Component	Technology
Frontend	TypeScript + React 18 + Vite + AWS Amplify Authenticator
Backend	AWS Lambda (Python 3.12) + API Gateway WebSocket
AI Models	Claude Sonnet 4.6 via Bedrock Inference Profile
Data Sources	CloudWatch Logs Insights, AWS X-Ray, Custom Evaluator Logs
Authentication	AWS Cognito (username/email support)
Infrastructure	AWS SAM (CloudFormation)
Observability	CloudWatch Logs + Structured Audit Trail

1.2 Two-Mode Chat Architecture

Mode	Function	Backend Target
Assistant Mode	Pricing queries, general help	AgentCore runtime (calculator/app.py) — direct pass-through
Analyzer Mode	Log analysis, trace deep-dives	LLM-driven pipeline (analyzer/app.py) — intent detection + data collection

2. Request Flow — Analyzer Pipeline

Every user question in Analyzer Mode passes through a six-stage pipeline before a response is returned. Each stage is audited to CloudWatch in real time.

1. Intent Detection — Claude classifies the question into one of 6 intent types and extracts entities (user names, trace IDs, time windows). Max 320 output tokens to keep planning fast.
2. Data Collection — Based on intent, the system decides which sources to query (runtime logs, evaluator logs, X-Ray). Bounded at 5,000 events per query.
3. Context Slicing — Data is packaged into a payload appropriate for the detected intent. Compact rows for listings; full objects for deep dives.
4. Model Invocation — Claude Sonnet 4.6 receives the prepared context and generates a natural language answer (dynamic token budget).
5. Chunked Response — Answer is streamed in chunks over the WebSocket connection to the frontend (30 KB max per frame).
6. Audit Trail — All decisions, payloads, and outcomes are logged as structured JSON to CloudWatch for full observability.

DESIGN PRINCIPLE: The pipeline is intentionally adaptive — listings skip evaluator fetches for speed, while deep-dive and anomaly intents pull X-Ray segment data. This avoids unnecessary latency and token cost on simple queries.

3. Intent Detection (LLM-Powered)

Intent detection is the first and most critical stage of the pipeline. It drives every downstream decision about data collection and context packaging. The planning model is Claude Sonnet 4.6 with a hard 320-token output limit.

3.1 Intent Types

Intent Type	When Used	Data Strategy
summary	High-level performance reports, error summaries	Sample top 20 sessions, skip X-Ray
listing	Enumeration of session IDs, trace IDs, users	Compact row format, skip evaluator
deep_dive	Detailed analysis of a specific session or trace	Full session objects + X-Ray segments
comparison	Side-by-side comparison of users, sessions, time ranges	Full session objects + X-Ray segments
anomaly	Find outliers, errors, or unexpected patterns	Full session objects + X-Ray segments
other	Fallback for vague or conversational questions	Full session objects with all details

3.2 Intent Detection Prompt — Example

```
Input:  "give me a short report for nikhil"
Output: {
  "intent_type": "summary",
  "data_scope": "user_window",
  "lookback_hours": 720,
  "detail_level": "low",
  "user_names": ["nikhil"]
}
```

3.3 Retry Pass for Ambiguous Queries

If the initial classification returns an ambiguous or low-confidence result, the system performs a second planning pass with a more directive prompt. This ensures vague or conversational questions are still routed correctly without returning an error to the user.

4. Adaptive Data Collection

Data collection is gated by intent type to minimise latency. The system queries only the sources it actually needs, and all queries are bounded to prevent runaway cost.

4.1 Data Source Selection by Intent

Data Source	listing	summary	deep_dive / anomaly / comparison	other
Runtime Logs	Yes	Yes	Yes	Yes
Evaluator Logs	No	Yes	Yes	Yes
X-Ray Traces	No	No	Yes	No

4.2 Query Limits & Time Budgets

Limit	Value	Purpose
Lambda Timeout	22 seconds	CloudWatch query + model invoke + WebSocket response combined
Max Query Results	5,000 events	Runtime + evaluator events per CloudWatch Insights query
Max Token Budget	20,000 tokens	Prompt + context + answer combined
WebSocket Payload	30 KB	Response chunks per WebSocket frame sent to frontend
Continuation Token	4 KB max	Prevents WebSocket frame overload on paginated results
Context Truncation	42,000 chars	Hard fallback limit when payload would exceed token budget

5. Context Slicing

Different intents receive different context payloads to Claude. This is controlled by _context_for_model_prompt() in analyzer/app.py. Sending only what is needed keeps token usage low and answers focused.

Intent + Data Type	Context Sent to Claude
listing + session_ids	[session_rows] only — compact row format
listing + trace_ids	[trace_rows] only — compact row format
summary	[session_rows] + top 20 sessions sampled
deep_dive / anomaly / comparison	[full session objects with all detail fields]
other (fallback)	[full session objects with all details]

5.1 Pagination with Continuation Tokens

When results exceed 30–45 rows per page, the system splits them and issues a Base64-encoded continuation token containing the original question, inferred plan, and current offset. The user can ask "show me more" and the system resumes from the correct position without re-running the full analysis.

6. Audit Logging

Every analysis decision is logged as a structured JSON audit entry to CloudWatch, giving the team full real-time visibility into the system's reasoning — essential for debugging and demo walkthroughs.

6.1 Log Locations

Resource	Value
Audit Log Group	/aws/sensei/analyzer-websocket
Audit Log Stream	analyser-websocket
Runtime Log Group	/aws/bedrock-agentcore/runtimes/my_agent1-EuvQcG3t0u-DEFAULT
Runtime Log Filter	agent-traces
Evaluator Log Group	/aws/bedrock-agentcore/evaluations/results/evaluation_quick_start_1773400924069-RMD3JBHdQM

6.2 Audit Stages

Stage	What Is Logged
request_received	User question text + WebSocket session ID
plan_inferred	Detected intent type + extracted entities (users, trace IDs, time window)
data_collection_strategy_selected	Which sources will be fetched — runtime, evaluator, X-Ray flags + detail level
context_sent_to_model	Keys included in the final prompt payload sent to Claude
analysis_step_before_model_invoke	Comprehensive pre-answer summary — intent, entities, data shape, token estimate
response_sent	Final answer text + continuation token (if paginated)

6.3 Example Audit Entry

```
{
  "ts": "2026-04-07T12:00:00Z",
  "stage": "data_collection_strategy_selected",
  "request_id": "req-abc123",
  "fields": {
    "intent_type": "listing",
    "will_fetch_evaluator": false,
    "will_fetch_xray": false,
    "detail_level": "medium"
  }
}
```

7. AWS Resources Reference

Resource	Identifier / ARN / Value
AgentCore Runtime ID	my_agent1-EuvQcG3t0u
AgentCore Runtime ARN	arn:aws:bedrock-agentcore:us-east-1:636052469006:runtime/my_agent1-EuvQcG3t0u
Evaluator Run Family	evaluation_quick_start_1773400924069
Runtime Log Group	/aws/bedrock-agentcore/runtimes/my_agent1-EuvQcG3t0u-DEFAULT
Runtime Log Stream Filter	agent-traces
Evaluator Log Group	/aws/bedrock-agentcore/evaluations/results/evaluation_quick_start_1773400924069-RMD3JBHdQM
Analyzer Audit Log Group	/aws/sensei/analyzer-websocket
Analyzer Audit Stream	analyser-websocket
Price Catalog S3 Bucket	my-agent1-price-catalog-636052469006-us-east-1
Analyzer Model ID	arn:aws:bedrock:us-east-1:636052469006:inference-profile/global.anthropic.claude-sonnet-4-6

8. Frontend Setup (TypeScript+ Vite)

8.1 Key Source Files

File / Directory	Purpose
src/components/chat/ChatComposer	Message input bar with send button
src/components/chat/MessageList	Renders streamed assistant messages
src/components/chat/AnalyzerHelpPanel	Sample query suggestions panel
src/pages/AssistantChatPage	Assistant mode chat page
src/pages/AnalyzerChatPage	Analyzer mode chat page
src/hooks/useChat.ts	WebSocket connection + streaming state management
src/services/websocketClient	Raw WebSocket client with reconnect logic
src/services/auth	Cognito auth helpers via Amplify
src/config.ts	Environment variables + Amplify configuration
src/types/chat.ts	TypeScript interfaces for messages and WebSocket events

8.2 Authentication

Authentication is handled by AWS Cognito via the Amplify Authenticator component. Both username and email login are supported. The Cognito JWT token is attached to WebSocket connection handshake headers for backend authorisation.

8.3 Build Commands

```
cd frontend
npm install
npm run dev          # Development server at http://localhost:5173
npm run build        # Production build output to frontend/dist/
```

9. Backend — Lambda Functions

9.1 Analyzer Lambda (functions/analyzer/app.py)

This is the core of the system at approximately 1,750 lines. It implements the full six-stage pipeline: intent detection, data collection, context slicing, model invocation, chunked WebSocket streaming, and audit logging.

Key Functions

Function	Approx. Line	Role
_infer_analysis_plan()	L587	Calls Claude with planning prompt, parses JSON intent output, handles retry pass
_is_listing_intent()	L1178	Gate function — decides whether to skip evaluator fetch
_context_for_model_prompt()	L1072	Packages context payload based on intent type and collected data
_audit()	L87	Writes structured audit log entry to CloudWatch
lambda_handler()	L~1700	WebSocket event router — connect, disconnect, message, $default

9.2 Calculator Lambda (functions/calculator/app.py)

A thin wrapper that proxies requests from the Assistant Mode frontend directly to the AgentCore pricing runtime. No intent detection or data collection — simple pass-through with response streaming.

9.3 Lambda Environment Variables

Variable	Value / Description
RUNTIME_LOG_GROUP	/aws/bedrock-agentcore/runtimes/my_agent1-EuvQcG3t0u-DEFAULT
EVALUATOR_LOG_GROUP	/aws/bedrock-agentcore/evaluations/results/evaluation_quick_start_1773400924069-RMD3JBHdQM
ANALYZER_MODEL_ID	arn:aws:bedrock:us-east-1:636052469006:inference-profile/global.anthropic.claude-sonnet-4-6
DEFAULT_ANALYSIS_LOOKBACK_HOURS	720 (30-day default lookback window)
ANALYZER_REQUEST_BUDGET_SECONDS	22 (total time budget per request across all pipeline stages)

10. Deployment — AWS SAM

10.1 Build & Deploy

```
# Frontend
cd frontend && npm run build
```

```
# Backend
sam build
sam deploy
```

For first-time deployment, use guided mode which prompts for stack name, region, and S3 bucket:

```
sam deploy --guided
```

> **NOTE:** Deployment configuration is persisted in samconfig.toml. Subsequent deploys use these saved values automatically. Review this file carefully before deploying to a new environment.

10.2 AgentCore Runtime

The pricing agent (my_agent1.py) is deployed as a separate AgentCore runtime configured via .bedrock_agentcore.yaml. This is the backend for Assistant Mode and is fully independent of the SAM stack.

11. Sample Queries & Expected Behavior

Use these queries when testing or demoing the system. They are organised by intent type to validate correct pipeline routing.

11.1 Summary Queries

- "give me a short report for nikhil"
- "what's the overall performance this week?"
- "summarize errors for user alice"

11.2 Listing Queries

- "list out all session ids"
- "show me all trace ids for user bob"
- "give me more info about sessions for alice"

11.3 Deep Dive / Anomaly Queries

- "compare sessions for user alice vs bob"
- "deep dive into trace-id <ID> — why was it slow?"
- "find anomalies in the last 48 hours"

11.4 Contextual Follow-ups

- "show me more" — resumes previous paginated listing from saved continuation token
- "explain that trace" — references prior trace ID from current analysis session

> **TIP:** Continuation tokens expire at the end of a WebSocket session. If the user disconnects and reconnects, they must re-run the original query to start a fresh paginated result set.

12. Code Quality & Known Issues

Area	Status	Detail
Active Functions	Clean	49 active functions — no dead code identified
Data Slicing	Clean	Intent-driven context packaging follows DRY principle
Parameter Bounds	Clean	All queries bounded (100-item limits, token budgets, timeouts)
Error Handling	Clean	Graceful fallbacks at each pipeline stage
Utility Duplication	Minor Issue	3 utility functions duplicated across calc and analyzer lambdas — refactor opportunity
Large Context Payloads	Minor Issue	Context payload can reach ~11K tokens for large datasets — within budget but worth monitoring

13. Troubleshooting Guide

Symptom	Diagnosis & Resolution
WebSocket connection drops	Check API Gateway WebSocket stage deployment. Verify Cognito JWT is valid and not expired before connection attempt.
"No data found" for all queries	Verify RUNTIME_LOG_GROUP and EVALUATOR_LOG_GROUP env vars point to correct CloudWatch log group paths. Test with aws logs tail.
Intent always classified as 'other'	Check the planning model response in audit logs (plan_inferred stage). Confirm ANALYZER_MODEL_ID is set correctly.
Lambda timeout (504)	Reduce DEFAULT_ANALYSIS_LOOKBACK_HOURS or lower max query results. Check CloudWatch Insights query execution time separately.
Continuation token too large	Context for the page exceeds 4 KB limit. Reduce rows per page or increase compression of row format.
X-Ray data missing in deep dives	Confirm Lambda execution role has xray:GetTraceSummaries and xray:BatchGetTraces permissions. Verify X-Ray tracing is enabled.
sam deploy fails	Run sam build first. Check samconfig.toml for correct region and S3 bucket. Ensure AWS CLI credentials are configured.
Evaluator logs empty	Verify evaluator Lambda is writing to the correct log group with the expected JSON schema.

13.1 Useful Log Tail Commands

```
# Analyzer WebSocket Lambda audit logs
aws logs tail /aws/sensei/analyzer-websocket --follow
```

```
# Filter for plan_inferred audit stage only
aws logs tail /aws/sensei/analyzer-websocket --follow \
  --filter-pattern '{ $.stage = "plan_inferred" }'
```

```
# Runtime agent traces
aws logs tail /aws/bedrock-agentcore/runtimes/my_agent1-EuvQcG3t0u-DEFAULT --follow
\
  --log-stream-name-prefix agent-traces
```

14. Roadmap

Current (v1.0) — Production Ready

- LLM-powered intent detection with 6 intent types
- Adaptive data collection from CloudWatch Logs, X-Ray, and evaluator logs
- Real-time WebSocket streaming of analysis responses
- Pagination with Base64 continuation tokens
- Full structured audit trail in CloudWatch
- AWS Cognito authentication

Planned Improvements

7. Extract Shared Utilities — Move duplicated helpers to backend/functions/shared_utils.py
8. Add Caching Layer — Redis/ElastiCache for frequent user queries to reduce CloudWatch Insights costs
9. Multi-Model Routing — Switch between Claude Sonnet / Haiku by intent complexity for cost efficiency
10. Custom Metrics Framework — Extensible scoring system for evaluator logs
11. Trace Comparison View — Side-by-side X-Ray segment walkthrough diffs

15. Pre-Deployment Checklist

Item	Status
RUNTIME_LOG_GROUP env var set and pointing to correct log group	[ ]
EVALUATOR_LOG_GROUP env var set and pointing to correct log group	[ ]
ANALYZER_MODEL_ID set to correct Bedrock inference profile ARN	[ ]
sam build completes without errors	[ ]
sam deploy completes and outputs WebSocket API URL	[ ]
Cognito User Pool created and frontend config.ts updated with Pool ID	[ ]
Lambda execution role has CloudWatch Logs, X-Ray, and Bedrock permissions	[ ]
End-to-end test: login → summary query → listing query → deep dive	[ ]
Audit logs visible in CloudWatch after test queries	[ ]
WebSocket pagination tested ("show me more" returns next page)	[ ]
Lambda timeout confirmed at 22 seconds or higher	[ ]
No secrets or credentials committed to the repository	[ ]